

Edge Triage of Heterogeneous Urban Signals: A Fog-to-Cloud Operations Board for a Two-Zone Smart City

Mohammed Hassan Ahmed
Student ID: X25100963
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25100963@student.ncirl.ie

Abstract—A city district produces its operational picture from signals that share almost nothing: vehicle flow counted per minute, fine-particulate air quality in micrograms, noise in decibels, parking occupancy as a percentage, and ambient light in lux. An operations desk that wants one live verdict across all five must reconcile scales that differ by orders of magnitude, and doing so by streaming every raw reading to a distant region wastes bandwidth on values that individually rarely matter. This paper presents a continuous monitoring pipeline that answers that problem with edge triage. Five signal types are simulated across two city zones by ten independent processes, a fog gateway on the edge host reduces each zone-and-signal stream to a per-window summary and evaluates civic threshold rules locally, and only the finished summaries cross the network. A scalable serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB ingests those summaries and serves an operations board through Amazon API Gateway, with the static frontend delivered from Amazon S3. Every tier is argued against the alternatives considered and set aside, and the system was deployed to a real Amazon Web Services account rather than only a local emulator. An automated suite of 62 tests passes, and verification against the live deployment confirmed each pipeline health indicator reporting healthy, fresh windows arriving within a few seconds, both zones rendering all five signals, and the incident banner correctly reporting a quiet city.

Keywords—smart city, urban sensing, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB

I. INTRODUCTION

A modern city district is instrumented, but its instruments do not agree on units, cadence, or what counts as a problem. A traffic counter reports tens to a few hundred vehicles a minute; an air-quality probe reports single-digit to low-tens micrograms of fine particulate; a sound meter sits in the fifties and sixties of decibels; a bay sensor reports occupancy as a percentage; a light meter swings across four orders of magnitude between night and midday. A supervisor watching a district does not want five raw feeds on five scales. They want a fused, current answer to a simple question: is any zone in trouble right now, and if so, on which signal. Research on the Internet of Things for smart cities frames exactly this as the core data-handling challenge, dense heterogeneous sensing

whose value lies in timely interpretation rather than in the raw stream [1], [2].

The distributed-systems question is where that interpretation happens. Sending every sample from every sensor to a distant cloud region is wasteful in two directions at once: it spends bandwidth on readings that, taken alone, seldom change a decision, and it defers the one moment that matters, the instant a signal crosses a civic limit, until after a network round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [3], [4], and it is one expression of the wider movement of computation back toward the network edge [5], [6]. Behind that edge tier, the cloud contributes the elastic, pay-per-use model that removes the need to pre-size a server for peak load [7], so it can absorb bursts and cost nothing while idle.

This project builds that two-tier design for a scaled but structurally faithful district: two zones, designated zone-1 and zone-2, each carrying the same five signal types, running as ten independent sensor processes. A fog gateway on the edge host receives their readings, groups each zone-and-signal pair into a fixed time window, reduces the window to summary statistics, evaluates a civic rule against the summary, and forwards only the finished window and any alert it raises. A serverless cloud pipeline persists each window, and an operations board folds the persisted history into a per-zone card view alongside a cross-zone trend.

The work pursues four objectives, each demonstrated on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous signals at sampling and dispatch rates that are configurable rather than fixed in code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a board that converts stored windows into an operational reading a supervisor could act on without inspecting raw values. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account

and verification against the live endpoint. Scope is deliberately held to these layers at a two-zone scale rather than a full multi-district network. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice, including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limitations, and the directions the work could take next.

II. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog gateway sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one signal type in one zone. On every sampling tick a process advances a bounded random walk: it adds a step drawn uniformly from a small symmetric range to its previous value and clamps the result to a physically plausible band for that signal, so consecutive readings stay close together like a real instrument trace rather than independent noise. This matters for the whole pipeline, because it is what makes a window's minimum, maximum, and average, and the board's trend line, resemble genuine city telemetry instead of static. The plausible bands differ sharply by signal, which is the heterogeneity the design has to carry: vehicle flow ranges across hundreds per minute, ambient light across tens of thousands of lux, and noise across a few tens of decibels.

Two independent, separately configurable intervals drive each process. One governs how often a fresh value is sampled, defaulting to two seconds; the other governs how often the accumulated readings are dispatched to the gateway, defaulting to ten seconds. Keeping the two genuinely separate, rather than aliasing one to the other, lets a fast-moving quantity be sampled often while still being sent in economical batches. Each dispatch is a compact JSON object naming the signal type, the zone, and the unit, and carrying the batch of timestamped values accumulated since the previous dispatch. At the default rates a dispatch carries roughly five readings and amounts to a few hundred bytes on the wire, well under a kilobyte. The process posts it over HTTP to the gateway's ingest endpoint, so the input interface is an ordinary HTTP call the sensors make rather than a shared file or a direct database write.

The fog gateway is a Java service that runs as its own container, distinct from the sensor processes and from the cloud functions, on the edge host. It does real work before anything leaves the edge, not a pass-through relay. Incoming readings are grouped by zone and signal into the current time window under a concurrency scheme that lets many writers add to a window while a single sealing operation swaps in a

fresh, empty window and drains the sealed one, so a reading that arrives as a window closes is never lost and never blocks. When a window closes, each non-empty group is reduced to a summary carrying the count, minimum, maximum, and average, and that summary, not any raw reading, is evaluated against the district's civic rules: vehicle flow above 180 a minute raises a congestion risk, fine particulate above 35 micrograms raises an air-quality alert, noise above 75 decibels raises a noise violation, parking occupancy above 95 percent raises a parking-full condition, and ambient light below 5 lux raises a low-visibility alert. The rule set is exposed for display through a separate declarative catalogue, kept deliberately independent of the code that evaluates it, so the board can disclose the thresholds without the disclosure ever driving evaluation.

The windowing mechanism itself is where most of the gateway's care goes, because readings arrive continuously from ten sources while windows have to close cleanly on a fixed boundary. The live window is a ledger held behind a single atomic reference, and within it each zone-and-signal group has its own accumulator in a concurrent map. An arriving reading briefly marks itself as an active writer, adds its values to the relevant accumulator, and clears the mark. When the boundary timer fires, the sealing operation swaps a fresh empty ledger into the atomic reference in one step, so new readings immediately land in the new window, and then waits for any writers still in flight against the old ledger to finish before it reduces and dispatches it. The effect is that no reading is dropped at a boundary and no reading is double-counted across two windows, and the hot path never takes a lock that a slow dispatch could hold. This is the detail that lets the edge stay both correct and responsive while five signals on very different cadences all fold into the same window structure.

The gateway's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries into a single send to Amazon SQS, which keeps the number of send operations proportional to the number of distinct signals rather than to the sampling rate. The failure behaviour on that send is a deliberate design choice and is worth stating plainly: the window is sealed and retired from the live buffer before the send is attempted, and the send is wrapped so that a failure is logged and the cycle continues rather than crashing the gateway or blocking new readings. Because the window has already been retired, a failed send drops that one window rather than stalling the edge, a bias toward keeping the gateway live and current that suits a monitoring workload where the next window is only seconds away and a momentarily missed summary is preferable to a wedged edge.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the gateway's summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the board. Splitting the write path from the

Ingest path (solid): edge → queue → function → store. Serving path: function → gateway → browser, assets from S3. Dashed: the dashboard's four live read sources.

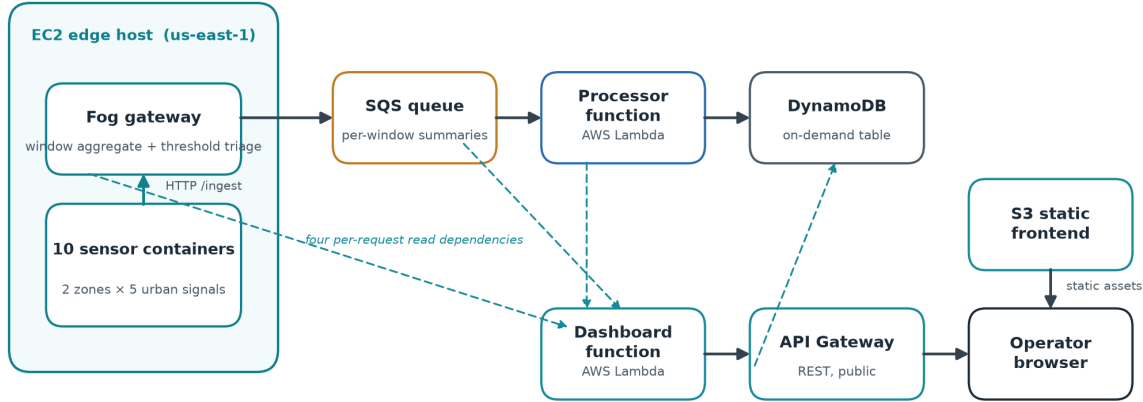


Fig. 1. Deployment topology. Ten sensor containers and the fog gateway share one Amazon EC2 edge host. Solid arrows are the ingest path, from the gateway through Amazon SQS and the processor function into the Amazon DynamoDB table. The serving path runs from the dashboard function through Amazon API Gateway to the browser, with static assets delivered from Amazon S3. The dashed edges are the four sources the dashboard function reads on each request: the fog gateway’s health and threshold endpoints, the queue’s depth, the processor function’s deployment state, and the stored windows.

read path is deliberate: ingestion and querying scale and fail independently, so a surge of windows cannot slow the board and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The gateway could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the gateway stalls or sheds data while the pressure propagates back toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer’s behalf, and lets the gateway treat a summary as delivered the moment it is enqueued [8]. On a window cadence the small added latency is immaterial, while the decoupling and durability are exactly what an unreliable edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved up front [7]. The recognised cost is the cold start when a function has been idle [9]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the board’s own polling keeps the interface function warm. The processor function is triggered by the queue and neither holds state between invocations nor is billed while idle.

One batched send instead of one send per summary. When a window closes, several zone-and-signal groups usually close together, and each is a separate summary. The gateway could send them one at a time, but it instead combines every summary from a flush cycle into a single batched send to the queue. The reason is that the cost that matters downstream is the number of send operations and the number of function

invocations they trigger, not the number of readings, and a single shared timer with per-signal bands can produce many small windows at once. Batching keeps that cost proportional to the number of distinct signals closing in a cycle rather than multiplying it out, and it is bounded by the queue’s own per-batch limit, above which the gateway simply issues more than one batch. This is the same reduction logic the fog aggregation applies, carried one level further, so that both the volume of data and the number of calls stay tied to how many signals exist rather than to how fast they are sampled.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known ahead of time: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and avoids the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service carries forward [10], [11]. The key design follows from the query. Each stored window is keyed on its signal type as the partition and on a sort value that leads with the window’s closing timestamp and then the zone, so windows for one signal are already ordered chronologically within a single partition and the newest are read with one bounded, single-partition query rather than a scan.

C. Serving Layer and Security Posture

The board reaches the read function through a managed gateway that publishes a public REST endpoint. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. The read function exposes a small, read-only surface of five endpoints: a zones

endpoint giving the newest window for every signal in both zones, a readings endpoint returning a recent series for one signal to feed the trend charts, a health endpoint, a backend-statistics endpoint, and a thresholds endpoint that forwards the gateway's published rules. The health response is assembled from four live sources at once, the store for the freshest windows, the queue for its depth, the processor function for its deployment state, and the gateway for its own reachability and thresholds, which are the dashed edges in Fig. 1, and it returns four separate flags together with the age of the newest window.

On the page, each zone is a card of five signal rows drawn against native meter elements, each meter scaled to the plausible band for its signal so a value reads as a position rather than a bare number, with a headline that reports how many zones are reporting and how many incidents are active, an incident banner that names any firing condition, and a set of cross-zone trend charts placing the two zones side by side. The incident count is derived on the client from the same window data the cards show, so the banner and the cards can never disagree, and a zone that has stopped reporting is distinguished from a zone reporting normal values rather than being shown as an empty card. The trend charts are drawn with a charting library kept in the repository so the page has no external dependency at load time, and each chart is redrawn in place on every cycle rather than being torn down and rebuilt, so the view updates without flicker. A fixed browser-side timer re-fetches the zones view, health, and backend statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it, substituted with the real endpoint when the assets are uploaded and read once by the script; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog gateway's health and thresholds endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, so what they disclose is bounded to what the board already renders, while the cloud tier relies on the managed services' own access control and on scoping each function to the queue, table, and gateway it needs.

III. IMPLEMENTATION

The sensor processes and the fog gateway are written in Java and packaged as containers; the two cloud functions are Java on the Java 17 runtime. The gateway serves its ingest, health, and threshold endpoints on the Java Development Kit's built-in HTTP server rather than a web framework, keeping the edge component dependency-light. JSON is handled with Jackson on the cloud side and with direct assembly on the sensor side, the AWS SDK for Java carries the queue and table calls, and the frontend renders its trends with a locally vendored copy of Chart.js. The edge tier is orchestrated with Docker Compose,

the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 62 tests: 3 over the sensor process, 16 over the fog gateway, 10 over the processor function, and 33 over the dashboard function. They run without any cloud connection by exercising the units against hand-written fake AWS clients, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The fog tests are the densest, and deliberately so, because the edge is where the concurrency lives: they cover the windowing and sealing under concurrent writers, the summary arithmetic on known inputs, and each civic rule at its limit and one step either side of it, so a boundary that should fire and a value that should not are both pinned down. The processor tests cover the normalisation of a queued summary into a stored item, including the composite sort value that orders windows within a partition. The dashboard tests cover the per-zone assembly from stored windows, the recent-series query that feeds the trend charts, the four health checks, and the request routing the deployed function performs, since that routing is code the local server never runs and so is the part most exposed to a deployment-only mistake. The suite runs on every push through GitHub Actions, so a change that breaks a rule boundary, a query shape, or the routing fails in continuous integration rather than after deployment.

Testing the fog against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service actually behaves under load, so behaviours such as batched-send size limits and the queue's own retry semantics were reasoned about from the service documentation and then confirmed against the live deployment rather than in unit tests. The alternative, standing up a local emulator for every test, was rejected as slower and still not the real service, and the gap it leaves is exactly what the live verification below is for.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions, the table, the gateway with its single catch-all route, and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog gateway and the ten sensor containers under Docker Compose; the managed queue, the two functions, the table, the managed gateway, and the frontend bucket make up the cloud side. The deployed table, queue, functions, and buckets all carry a common prefix so the

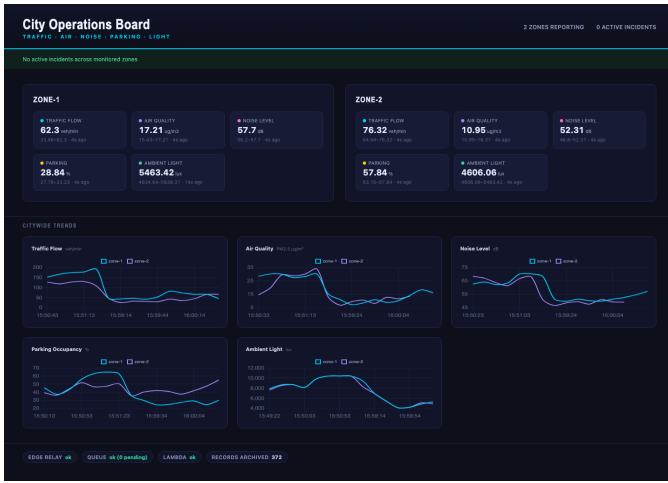


Fig. 2. The deployed operations board reading from the live endpoint. Each zone shows its five signals with current value, recent range, and freshness; the header reports zones reporting and active incidents; the cross-zone trend charts place both zones side by side; and the footer strip shows the four pipeline indicators and the stored record count.

deployment is easy to identify and to remove as a unit, and the read function is fronted by a REST gateway that publishes the public endpoint the board calls. A fixed public address in front of the edge host means the read function can always reach the gateway's health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed board during that check. Within about a minute of start-up the pipeline reported healthy end to end: the four health indicators, edge relay, queue, function, and freshest-window recency, all read healthy, and the newest window was a few seconds old on each poll. Both zones rendered all five signals with their current value, recent range, and freshness, the headline reported two zones reporting and no active incidents against a quiet simulated city, and the five cross-zone trend charts filled and advanced as new windows arrived. The backend-statistics endpoint showed the stored record count climbing across successive polls rather than a static number, which confirms the ingestion function was draining the queue and writing on an ongoing basis. Reaching the board from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

IV. CONCLUSION

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five heterogeneous urban signals across two zones into a single live operational reading, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each zone-and-signal stream and evaluates civic rules locally, so only compact summaries cross the network and a threshold breach is flagged in the window it occurs. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts

and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the board's one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 62 automated tests guarding its behaviour and continuous integration running them on every change.

Two reflections stand out from building it. The first is that the value of the fog tier is easiest to see precisely because the signals are heterogeneous: reducing five streams on five different scales to uniform per-window summaries at the edge is what lets one small, uniform record type flow through the queue, the function, and the table, and what keeps the downstream cost proportional to the number of signals rather than to how fast they are sampled. The second is that deploying to a real account is a different claim from passing tests, because the failure modes that matter at the edge, a send that fails while a window has already been retired, a cold function on the read path, a cross-origin call from a static bucket, are exactly the ones a local fixture never exercises, and each had to be reasoned about against production behaviour rather than assumed away.

The work has clear limits. It runs at a two-zone scale with simulated sensors, its civic thresholds are fixed rather than learned or configurable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to more zones and real sensor feeds, to move the fixed thresholds to a managed configuration an operator could adjust without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history for trend analysis beyond the live view. None of these changes the shape of the pipeline, which is the point: the edge-triage design carries from two zones to many without rethinking the tiers.

REFERENCES

- [1] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for Smart Cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.
- [2] C. Perera, A. Zaslavsky, P. Christen, and D. Georgakopoulos, "Sensing as a Service Model for Smart Cities Supported by Internet of Things," *Trans. Emerging Telecommunications Technologies*, vol. 25, no. 1, pp. 81–93, 2014.
- [3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.
- [4] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.
- [5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [6] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [7] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [9] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.

- [10] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
- [11] M. Elhemali et al., “Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service,” in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.
- [12] R. Mahmud, R. Kotagiri, and R. Buyya, “Fog Computing: A Taxonomy, Survey and Future Directions,” in *Internet of Everything*, Springer, 2018, pp. 103–130.
- [13] Amazon Web Services, “Amazon Simple Queue Service Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [14] Amazon Web Services, “AWS Lambda Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [15] Amazon Web Services, “Amazon API Gateway Developer Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>
- [16] IEEE, “Manuscript Templates for Conference Proceedings,” 2024. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>

The source code and deployment configuration are available at [GitHub repository URL].